Guion de presentación del Portal Vial IMSJ

De las necesidades del IMSJ y la ciudadanía a las decisiones técnicas

Duración sugerida 17 a 19 minutos

Estructura Necesidad y solución seguida de defensa técnica

Este guion parte de la necesidad general de la Intendencia de llegar a los jóvenes con información vial clara y vigente. Portal Vial aparece como la solución que conecta a la institución con la ciudadanía. Luego, cada necesidad concreta se vincula con una respuesta del portal o del panel, antes de justificar técnicamente cómo se construyó y se sostiene la solución.

Indicaciones entre corchetes son sugerencias de ritmo y no se leen en voz alta.

Mapa de la presentación

Acto 1 Necesidad y solución. Diapositivas 1 a 7. Parte de la necesidad general de la Intendencia y recorre las respuestas concretas del portal ciudadano y del panel IMSJ.

Acto 2 Las decisiones. Diapositivas 8 a 14. Justifica tecnologías, arquitectura, datos, seguridad, despliegue, operación y verificación.

Cierre. Diapositiva 15. Vuelve a la conexión entre la Intendencia y los jóvenes y resume la defensa técnica.

Tesis de la defensa: Portal Vial responde a una brecha de acceso y comunicación. La solución técnica es adecuada porque convierte necesidades reales en recorridos comprensibles, datos consistentes y una operación sostenible.

Acto 1 De la necesidad a la solución

Diapositiva 1 La necesidad general

Tiempo 55 segundos

En pantalla *Imagen de la Dirección de Tránsito intentando comunicarse con jóvenes. Frase central: necesitamos llegar.*

> [Mirar al público. Pausa breve.] La historia del proyecto comienza con una necesidad de la Dirección de Tránsito: llegar a los jóvenes con información vial clara, vigente y fácil de encontrar.
>
> El problema no era solamente producir información. Esa información podía quedar dispersa entre distintos canales, ser difícil de reconocer como vigente o no acompañar el momento en que una persona decide prepararse para obtener la libreta.
>
> Del otro lado estaba justamente esa persona joven, con una pregunta muy concreta: dónde estudiar y qué información tomar como referencia. La Intendencia necesitaba llegar; la ciudadanía necesitaba encontrar.

Idea que debe quedar La necesidad institucional de llegar se conecta con la necesidad ciudadana de encontrar.

Transición *Las dos perspectivas mostraban una misma brecha. La pregunta era cómo crear un punto de encuentro.*

Diapositiva 2 La solución general

Tiempo 55 segundos

En pantalla *Imagen puente de la presentación HTML y el nombre Portal Vial. Frase: la ciudadanía encuentra y Tránsito llega.*

> [Pausa breve antes de nombrarlo.] La respuesta general fue Portal Vial: un único punto de encuentro entre la ciudadanía y la Dirección de Tránsito.
>
> Para la persona joven, el portal concentra el recorrido de preparación, consulta e información. Para la Intendencia, el sistema incorpora un panel desde el cual puede mantener contenidos, atender consultas y organizar la operación.
>
> Esta es la idea que ordena toda la presentación: primero aparece una necesidad; después mostramos la respuesta concreta que construimos para resolverla.

Idea que debe quedar Portal Vial es la conexión general; sus secciones son respuestas a necesidades concretas.

Transición *Ahora podemos bajar de la solución general a las necesidades específicas del recorrido ciudadano.*

Diapositiva 3 Necesidad de prepararse

Tiempo 1 minuto

En pantalla *Dos pares necesidad y solución: estudiar → materiales; comprobar preparación → simulador.*

> La primera necesidad concreta aparece cuando una persona decide prepararse. Necesita consultar material de estudio sin recorrer fuentes dispersas. La respuesta es la sección de materiales, que reúne recursos accesibles desde el portal ciudadano.
>
> Después surge otra pregunta: cómo practicar antes del examen teórico. La respuesta es el simulador, que permite ensayar preguntas, elegir opciones y recibir una corrección. No reemplaza la prueba oficial; acompaña la preparación.
>
> En ambos casos la lógica es la misma: una necesidad del recorrido se transforma en una función visible y fácil de explicar.

Idea que debe quedar Necesidad: estudiar y practicar. Solución: materiales de estudio y simulador teórico.

Transición *Prepararse no elimina todas las dudas. El siguiente problema es encontrar una respuesta o formular una pregunta.*

Diapositiva 4 Necesidad de resolver dudas

Tiempo 1 minuto y 10 segundos

En pantalla *Dos pares conectados: duda frecuente → preguntas frecuentes; duda particular → consulta ciudadana y bandeja interna.*

> Durante la preparación aparecen dudas. Si la respuesta ya fue publicada, la persona necesita encontrarla sin iniciar un trámite. La sección de preguntas frecuentes organiza esas respuestas por categorías.
>
> Si la duda no está respondida, la necesidad cambia: hace falta un canal directo. El portal ofrece una consulta ciudadana. Esa consulta no queda aislada; llega a la bandeja del panel IMSJ, donde el personal puede leerla y gestionarla.
>
> Aquí se ve con claridad la conexión entre ambos lados. Una acción pública tiene una respuesta operativa dentro de la institución.

Idea que debe quedar Necesidad: obtener respuesta. Solución: FAQ para lo recurrente y consulta para lo particular.

Transición *Además de resolver dudas, la ciudadanía necesita información vigente y una forma de avanzar en el trámite.*

Diapositiva 5 Necesidad de informarse y reservar

Tiempo 1 minuto

En pantalla *Dos pares necesidad y solución: novedades vigentes → noticias; elegir horario → agenda y cupos.*

> La persona también necesita saber qué novedades siguen vigentes. La respuesta es la sección de noticias, que concentra información actualizada en el mismo lugar donde estudia y consulta.
>
> Cuando está preparada para avanzar, necesita elegir una fecha y un horario disponible. La agenda pública muestra franjas y cupos y permite realizar la reserva.
>
> El recorrido ciudadano queda así conectado: estudiar, practicar, resolver dudas, informarse y reservar. Pero para que ese recorrido funcione, la Intendencia necesita mantenerlo del otro lado.

Idea que debe quedar Necesidad: información vigente y acceso a un horario. Solución: noticias y agenda pública.

Transición *Cambiamos de perspectiva: pasamos de quien usa el portal a quien debe mantener y controlar la información.*

Diapositiva 6 Necesidad de gestionar y aprobar

Tiempo 1 minuto y 15 segundos

En pantalla *Panel IMSJ con gestión de contenidos y flujo editorial: borrador, pendiente, aprobado y publicado*.

La Intendencia no solo necesita publicar. Necesita gestionar el contenido, revisar quién puede modificarlo y separar la aprobación de la publicación.

La respuesta es el panel IMSJ con gestión editorial. Desde allí se mantienen noticias, materiales y preguntas frecuentes. El flujo distingue estados como borrador, pendiente, aprobado y publicado. Además, editar un contenido reinicia el ciclo para que un cambio no quede publicado sin nueva revisión.

Esta necesidad se volvió más precisa durante la validación con el cliente: aprobar y publicar debían ser responsabilidades diferentes. El panel convierte esa regla organizativa en un flujo visible del sistema.

Idea que debe quedar Necesidad: controlar la vigencia y la responsabilidad editorial. Solución: panel y flujo de aprobación.

Transición *El panel también debe responder a necesidades operativas que ocurren todos los días*.

Diapositiva 7 Necesidad de operar el servicio

Tiempo 1 minuto

En pantalla *Panel IMSJ con bandeja de consultas, agenda administrativa y reservas*.

> La Intendencia necesita recibir las consultas que genera el portal, revisar la agenda y administrar las reservas. La respuesta es la parte operativa del panel.
>
> La bandeja reúne los mensajes ciudadanos; la agenda administrativa permite observar franjas y cupos; y la gestión de reservas conecta lo que la persona hizo en el portal con el trabajo interno de Tránsito.
>
> Con esto se cierra el modelo necesidad y solución: la ciudadanía encuentra, consulta y reserva; la Intendencia publica, aprueba, recibe y gestiona.

Idea que debe quedar El portal y el panel no son productos separados: son las dos caras de un mismo recorrido.

Transición *Ya vimos qué problema resuelve cada parte. Ahora corresponde justificar por qué la solución está construida de esta manera*.

Acto 2 Las justificaciones técnicas

Diapositiva 8 Tecnologías y versiones

Tiempo 1 minuto y 20 segundos

En pantalla *Ubuntu Server 26.04 LTS, PHP 8.5, MySQL 8.4, Apache, Docker Engine y Compose v2.*

Ubuntu Server 26.04 LTS se eligió como host porque la aplicación no necesita un entorno gráfico. La administración puede realizarse por SSH, terminal y Cockpit, y una rama LTS reduce la frecuencia de cambios de distribución. La decisión no invalida Ubuntu 24.04 si la institución ya tuviera esa versión estandarizada.

PHP 8.5 se mantiene igual entre desarrollo, Composer y la imagen del contenedor. El argumento principal es la coherencia de una rama soportada, no una supuesta mejora automática de rendimiento. MySQL 8.4 se conserva como rama LTS y con InnoDB porque el dominio necesita relaciones, transacciones, claves foráneas y bloqueo de filas.

Apache y PHP se ejecutan en un mismo servicio web para resolver archivos, reescritura y ejecución con una configuración acotada. Docker y Compose describen los dos servicios, la red, los volúmenes y la salud sin introducir un orquestador de clúster.

Una precisión importante: etiquetas como php:8.5-apache o mysql:8.4 fijan una rama, pero no siempre el mismo binario. Para reproducibilidad deben registrarse la revisión y el digest efectivamente desplegados.

Idea que debe quedar Las versiones se justifican por soporte, coherencia y mantenimiento; no por ser simplemente las más nuevas.

Diapositiva 9 Arquitectura y organización del código

Tiempo 1 minuto y 20 segundos

En pantalla *Diagrama de capas: entrada, controladores, servicios, repositorios, modelos y núcleo.*

El backend es un monolito modular. El sistema comparte usuarios, datos y transacciones, por lo que dividirlo en microservicios habría agregado autenticación entre servicios, fallos de red y coordinación distribuida sin una ventaja demostrada para ocho módulos y un equipo pequeño.

Dentro del monolito sí hay separación. public/index.php centraliza la entrada y las rutas. Los controladores interpretan la solicitud y verifican identidad o rol. Los servicios aplican reglas y coordinan transacciones. Los repositorios concentran SQL. Los modelos y el núcleo reúnen comportamiento común.

Las dos interfaces usan HTML, CSS y JavaScript sin un framework de frontend. Para formularios, listados y contenido, esto mantiene visible la relación entre eventos y peticiones y evita una compilación adicional. La separación de carpetas facilita el mantenimiento, pero no se considera un control de acceso: la autorización siempre se vuelve a comprobar en el servidor.

Idea que debe quedar Monolito no significa código mezclado; significa una unidad de despliegue con responsabilidades internas separadas.

Diapositiva 10 Datos e integridad

Tiempo 1 minuto y 25 segundos

En pantalla *Ejemplo de dos personas intentando reservar el último cupo de una misma franja*.

La elección de MySQL se vuelve crítica en el caso del último cupo. Si dos personas consultan al mismo tiempo, ambas pueden ver disponibilidad. Por eso no alcanza con validar en el navegador ni con hacer una consulta antes del alta.

El servicio inicia una transacción, bloquea la franja, comprueba duplicación y capacidad, crea la reserva y recién entonces confirma. Las restricciones del esquema respaldan esa lógica. En operaciones administrativas, la modificación y su auditoría también se coordinan dentro de la misma transacción.

PDO y las consultas preparadas mantienen visible el SQL y separan la estructura de los valores. No se usa un ORM porque ocultaría parte del recorrido que el equipo necesita explicar, aunque el costo sea escribir y mantener más consultas.

El esquema inicial y las migraciones incrementales se mantienen separados: database.sql crea una instalación nueva y migrar.php actualiza una existente sin borrar volúmenes. Si la evolución crece, un historial numerado de migraciones sería el siguiente paso razonable.

Idea que debe quedar La integridad se protege en la transacción y en la base, no solo en la interfaz.

Diapositiva 11 Sesiones, permisos y archivos

Tiempo 1 minuto y 20 segundos

En pantalla *Tres bloques: token revocable, autorización en servidor y archivos fuera de la raíz pública*.

La sesión usa un token opaco aleatorio. El servidor guarda su hash, la cuenta asociada y el vencimiento. Esto permite revocar la sesión al cerrar, reemplazarla al iniciar nuevamente y comprobar que la cuenta siga activa. El costo es consultar la base en cada validación; para un único backend que ya depende de MySQL, ese costo simplifica el control.

Las contraseñas usan bcrypt con costo 12. El token se conserva en sessionStorage, lo que limita su persistencia a la pestaña, aunque no elimina el riesgo de XSS porque el JavaScript del origen puede leerlo.

Los adjuntos se validan por tamaño, tipo detectado y contenido básico, reciben nombres aleatorios y se guardan fuera de public. La descarga pasa por una ruta controlada. Esto reduce exposición, pero no equivale a un antivirus completo.

Además, el DocumentRoot se limita a public y las respuestas sensibles no se reutilizan en caché. Los recursos estáticos sí pueden conservarse por un período corto para reducir transferencias.

Idea que debe quedar La seguridad se construye con controles complementarios y límites explícitos; ninguna medida aislada resuelve todo.

Diapositiva 12 Despliegue y dimensionamiento

Tiempo 1 minuto y 20 segundos

En pantalla *Dos contenedores sobre una VM: app y db. Recursos iniciales: 2 vCPU, 4 GB RAM y 30 GB SSD o NVMe.*

Compose separa aplicación y base de datos. Así puede reconstruirse el contenedor de aplicación sin sustituir el volumen de MySQL. Los datos, adjuntos y caché también tienen ciclos de vida diferenciados. Un volumen aporta persistencia, pero no es una copia de seguridad.

El dimensionamiento inicial de 2 vCPU, 4 GB de RAM y 30 GB de disco parte de solicitudes breves, recursos estáticos, dos contenedores y hasta 90 usuarios con períodos de lectura. No se interpreta como 90 procesos PHP activos de forma permanente.

La VM debe ser ampliable y la hipótesis debe validarse con carga, latencia, errores y consumo real. Un 70 por ciento sostenido es un umbral para revisar capacidad, no una garantía de rendimiento.

Kubernetes no se incorporó porque el proyecto tiene un host y dos servicios. Sus ventajas de clúster no compensan aquí el costo administrativo. Si aparecieran múltiples nodos o exigencias fuertes de disponibilidad, esa decisión debería revisarse.

Idea que debe quedar La infraestructura se dimensiona como punto de partida medible, no como promesa definitiva.

Diapositiva 13 Operación, red y recuperación

Tiempo 1 minuto y 20 segundos

En pantalla *Secuencia operativa: salud, registros, backup diario, copia externa y restauración aislada.*

La operación incluye controles de salud, política de reinicio, rotación de registros, usuarios de servicio, raíz de solo lectura, tmpfs y capacidades reducidas. Estas medidas limitan fallos y privilegios, pero no sustituyen la actualización de imágenes.

La configuración se mantiene fuera del código mediante variables. La primera cuenta administrativa se crea por línea de comandos, evitando credenciales de demostración incorporadas al esquema.

El backup realiza un volcado lógico de MySQL, coordina SQL y adjuntos deteniendo la aplicación, comprime el conjunto y registra un manifiesto. La pausa es el compromiso de una estrategia simple y consistente. Cron reutiliza herramientas del servidor y la restauración se ensaya en otro proyecto Compose para reducir el riesgo de sobrescribir el entorno principal.

En red hay una salvedad que conviene expresar con claridad: los puertos publicados por Docker pueden eludir reglas habituales de UFW. La restricción efectiva debe verificarse con filtrado compatible con Docker y con pruebas desde redes autorizadas y no autorizadas.

Idea que debe quedar Operar significa poder observar, respaldar y reconstruir; no solo lograr que el sistema arranque.

Diapositiva 14 Pruebas, evidencia y límites

Tiempo 1 minuto y 10 segundos

En pantalla *Pirámide o lista: integración, auditoría API, análisis estático y exploración dinámica.*

La verificación combina técnicas porque cada una responde una pregunta diferente. La integración HTTP y MySQL comprueba contrato y persistencia. La auditoría API revisa accesos y entradas seleccionadas. El análisis estático y la exploración dinámica producen hallazgos que deben interpretarse.

El informe del 21 de octubre registró 192 comprobaciones sobre 46 endpoints. Ese resultado acredita la versión ensayada, no cualquier cambio posterior. Las modificaciones de categorías, validación e interfaz requieren su propia revalidación.

La defensa técnica también debe reconocer cuándo revisar la solución: más volumen puede exigir paginación y filtros de servidor; varios hosts, contadores compartidos; mayor disponibilidad, otra infraestructura; y más cambios de esquema, una herramienta formal de migraciones.

Idea que debe quedar La evidencia tiene alcance y fecha; una prueba histórica no reemplaza la verificación de la versión actual.

Transición *Con estas decisiones justificadas, el cierre vuelve a la idea que organizó todo el proyecto.*

Cierre

Diapositiva 15 Dos necesidades y una conexión

Tiempo 55 segundos

En pantalla *Retomar la imagen puente: la ciudadanía encuentra y la Intendencia llega*.

> [Pausa breve.] Empezamos con una necesidad de la Intendencia: llegar a los jóvenes. Del otro lado encontramos a una persona que necesitaba información clara para prepararse. Portal Vial conectó esas dos perspectivas.
>
> El PHP nativo hace visible el recorrido. Las capas ordenan las responsabilidades. MySQL y las transacciones protegen la consistencia. Los tokens revocables, los permisos y el manejo de archivos reducen exposición. Docker y Compose vuelven reconstruible el entorno. Los respaldos y las pruebas permiten sostenerlo y revisar sus límites.
>
> La conclusión es que cada parte puede explicarse desde una necesidad: estudiar, consultar, informarse, reservar, gestionar, aprobar u operar. Y cada decisión técnica se justifica por cómo sostiene esas respuestas. [Mirar al público.] Muchas gracias.

Idea que debe quedar Portal Vial permite que la ciudadanía encuentre y que la Intendencia llegue, con una solución técnica proporcional y sostenible.

Respuestas breves para la defensa

Por qué PHP nativo y no Laravel

Porque el objetivo fue hacer explícito el recorrido de una solicitud y facilitar la documentación y defensa. Se aceptó el costo de implementar rutas, validaciones y seguridad de forma visible. No se afirma que Laravel sea inadecuado.

Por qué un monolito y no microservicios

Porque el sistema comparte datos y transacciones, se despliega como una unidad y tiene ocho módulos. Microservicios agregarían autenticación interna, red y coordinación distribuida sin una necesidad demostrada.

Por qué MySQL y no archivos JSON

Porque el dominio necesita claves foráneas, restricciones, transacciones y bloqueo de filas para resolver concurrencia, especialmente en reservas.

Por qué token opaco y no JWT

Porque el token opaco puede revocarse de forma directa y permite comprobar el estado actual de la cuenta. El costo de consultar MySQL es aceptable en un único backend.

Por qué Docker Compose y no Kubernetes

Porque hay un host y dos servicios. Compose cubre red, volúmenes, configuración y salud con menor carga operativa. Kubernetes se reconsideraría con múltiples nodos o requisitos de alta disponibilidad.

La versión queda totalmente congelada

No. Las etiquetas fijan una rama, no siempre una revisión exacta. Para una entrega reproducible se deben registrar revisión y digest, actualizar parches y revalidar.

UFW alcanza para proteger el puerto publicado

No necesariamente. Docker puede alterar el recorrido del filtrado. La restricción debe verificarse con reglas compatibles con Docker y pruebas reales desde redes permitidas y no permitidas.

Un volumen Docker es un backup

No. El volumen conserva datos fuera del contenedor, pero puede borrarse o corromperse. El respaldo debe incluir base, adjuntos, configuración y código compatible, y debe existir una copia externa.

Las 192 pruebas garantizan la versión actual

Garantizan únicamente el alcance y la versión ensayada el 21 de octubre. Los cambios posteriores deben ejecutarse y documentarse nuevamente.